NAME:KRISHNAPRIYA P **REG NO:**192511279

COURSE CODE: CSA0613 **COURSE NAME :** Design Analysis of Algorithm

CO5 – AT1 --Constraint-Based Problem Solving

Q20. Independent Set Problem – Backtracking

1. Problem Understanding

Given a graph G = (V, E), the objective is to find the largest set of vertices such that no two selected vertices have an edge between them.

A set of vertices $S \subseteq V$ is called an independent set if:

$(u, v) \notin E$ for every $u, v \in S$

The largest such set is called the Maximum Independent Set.

2. Constraints for Vertex Selection

The following constraints must be satisfied:

Constraint 1: No Adjacent Vertices

Two selected vertices must not be connected by an edge.

Constraint 2: Unique Selection

A vertex can be selected at most once.

Constraint 3: Maximum Size

Among all feasible independent sets, the set with the largest number of vertices must be selected.

3. Constraint Analysis

Without constraints, for n vertices, there can be:

2^n

possible subsets.

For example, for 4 vertices:

$2^4 = 16$

possible subsets.

The independence constraint eliminates subsets containing adjacent vertices.

Thus, invalid subsets are rejected early, reducing unnecessary searching.

4. Backtracking-Based Solution

The algorithm considers each vertex one at a time:

1. Select a vertex.
2. Check whether it is independent of the current set.
3. If valid, include the vertex.
4. Recursively process the next vertex.
5. Backtrack by removing the vertex.
6. Also explore the possibility of excluding the vertex.
7. Store the largest valid set found.

5. Algorithm

Step 1: Start with an Empty Set

Initialize currentSet and bestSet as empty.

Step 2: Select a Vertex

Choose the next vertex.

Step 3: Check Constraint

Check whether the vertex has an edge with any vertex already in currentSet.

Step 4: Include Vertex

If no conflict exists, add the vertex and recursively continue.

Step 5: Backtrack

Remove the vertex after exploring that choice.

Step 6: Exclude Vertex

Explore the possibility of not selecting the vertex.

Step 7: Final Decision

When all vertices are considered, keep the largest independent set.

6. Pseudocode

ALGORITHM

MaximumIndependentSet(G, n)

1. bestSet ← ∅

2. FUNCTION Backtrack(v, currentSet)

3. IF v = n THEN update bestSet if currentSet is larger; RETURN

4. IF v has no edge with vertices in currentSet THEN

5. Add v to currentSet

6. Backtrack(v + 1, currentSet)

7. Remove v from currentSet

8. Backtrack(v + 1, currentSet)

9. CALL Backtrack(0, ∅)

10. RETURN bestSet

7. Pruning Conditions

The algorithm uses the following pruning condition:

Condition 1: Adjacent Vertex

If the current vertex is connected to any vertex in currentSet, it cannot be added.

Condition 2: Invalid Subset

Any subset containing two adjacent vertices is immediately discarded.

Condition 3: Size Bound

If the maximum possible size of the remaining vertices cannot improve bestSet, that branch can also be stopped.

Therefore, invalid branches are eliminated early.

8. Correctness Justification

The algorithm is correct because:

- It considers every vertex.
- A vertex is added only when it does not conflict with the current set.
- Backtracking explores both inclusion and exclusion possibilities.
- Invalid adjacent combinations are pruned.
- The largest valid set is stored as bestSet.

Therefore, the algorithm returns the maximum independent set.

9. Computational Complexity

For n vertices, there can be up to:

possible subsets.

Therefore:

Time Complexity: $O(2^n \cdot n)$ in the worst case.

Space Complexity: O(n) for the recursion stack and selected vertices.

10. NP-Complete Classification

The Independent Set decision problem asks:

“Does a graph contain an independent set of size at least k?”

This decision problem is NP-Complete because:

- A proposed independent set can be verified in polynomial time.
- The problem is NP-hard through reductions from other NP-Complete problems.

The Maximum Independent Set is the optimization version, where the goal is to find the largest independent set.

11. Conclusion

The backtracking algorithm finds the maximum independent set by systematically considering whether each vertex should be included or excluded. Constraint checking prevents adjacent vertices from being selected together, while pruning reduces unnecessary exploration. Although the problem has exponential worst-case complexity, backtracking provides a complete and correct solution for small graphs.